



TÓPICOS EM INFORMÁTICA

*Câmeras, viewport, telas, delta e
renderização contínua*



CAMERA

- Como podemos fazer para **movimentar** um personagem por uma cena?



CÂMERA

- Como podemos fazer para **movimentar** um personagem por uma cena?
 - Uma opção é **andar com a posição de todo o conteúdo do jogo** (todas as entidades gráficas) para simbolizar o movimento do personagem principal (o personagem que você controla).



CÂMERA

- Como podemos fazer para **movimentar** um personagem por uma cena?
 - Uma opção é **andar com a posição de todo o conteúdo do jogo** (todas as entidades gráficas) para simbolizar o movimento do personagem principal (o personagem que você controla).
 - A opção correta, contudo, é um utilizar **uma ou mais câmeras e mover a posição da câmera ao longo do cenário** (a posição dos itens do cenário permanece inalterada).
 - Mover somente a **câmera e o personagem** (o centro da câmera é a posição do seu personagem principal).

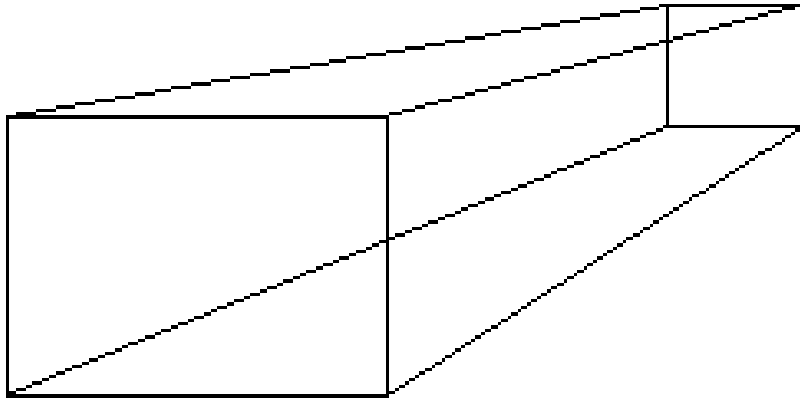
CÂMERA

- Em um jogo como super mario, por exemplo, o personagem principal tem a sua posição movimentada junto com a posição da câmera:

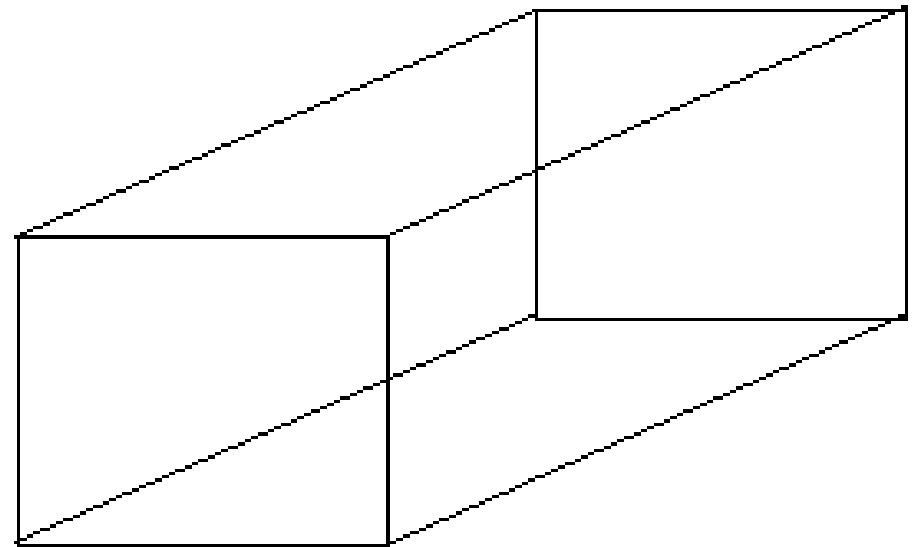


CÂMERA

- Podemos usar basicamente dois tipos de câmera na libgdx, a de **projeção perspectiva** (mundo real) e a **projeção ortográfica** (para jogos 2D ou para jogos 3D que tenham uma câmera parecida com 2D).



Perspective projection



Orthographic projection

CÂMERA

- Nesta figura temos o mesmo jogo/cenário sendo demonstrado por duas câmeras de **projeções diferentes**.
 - Ao lado esquerdo temos a **projeção perspectiva**.
 - Ao lado direito a **projeção ortográfica**.



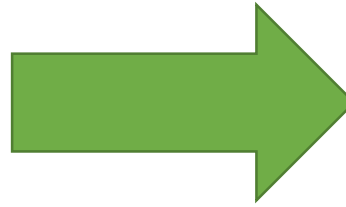


PROPRIEDADES DA CAMERA

- É possível realizar **diversas operações** com a câmera:
 - Mover (translação)
 - Rotacionar
 - Zoom
 - Mudar o viewport (seu campo de visão)
 - Projetar(**project**)/desprojetar(**unproject**) pontos do sistema de coordenadas de mundo para coordenadas de tela e vice-versa.

MOVIMENTO DE CAMERA

- Exemplo de movimentação da camera da esquerda pra direita.



ROTAÇÃO DE CAMERA

- Exemplo de rotação de câmera de 45 graus:



CAMERA ZOOM

- Exemplo de zoom no mesmo cenário:





PROJETAR PONTOS

- A projeção das cameras serve para traduzir um clique da tela (que tem coordenadas x, y) em coordenadas do mundo (depois da aplicação da câmera).
- Isso pode ser feito em ambos os caminhos (ida e volta). Ter uma coordenada de mundo e ver em qual pixel ela cairá na tela e ter a coordenada de um pixel e posteriormente saber qual posição esse pixel corresponde no mundo.
 - Isso é imprescindível para calcular **hitboxes**, onde o usuário clicou no mundo para posteriormente disparar uma ação, etc.

CAMERA ORTOGRÁFICA

```
OrthographicCamera cam;  
float w = Gdx.graphics.getWidth();  
float h = Gdx.graphics.getHeight();  
//iniciamos uma câmera ortográfica com altura e largura  
cam = new OrthographicCamera(30, 30 * (h / w));  
//setamos a posição central da câmera  
cam.position.set(cam.viewportWidth / 2f, cam.viewportHeight / 2f, 0);  
//atualizamos a transformação  
cam.update();  
  
...  
  
//aplicamos a câmera ao batch  
batch.setProjectionMatrix(cam.combined);
```



EXERCÍCIO

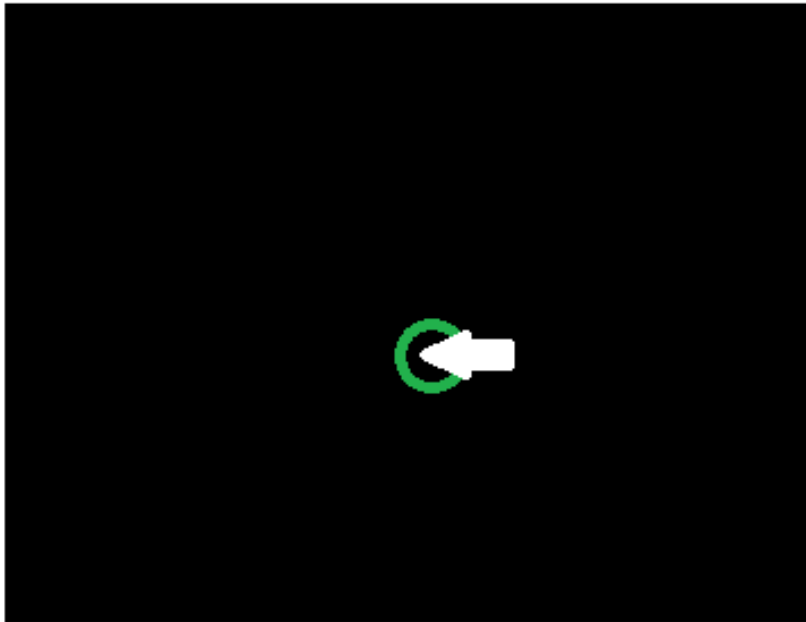
- Pegue o código do seu jogo de exemplo e adicione um **personagem principal** e pelo menos outros **2 elementos** da cena além do fundo.
- Adicione uma **câmera e movimento a câmera e o personagem principal** (que está sendo controlado) a partir da movimentação do teclado.

CAMERA VS VIEWPORT

- Camera e viewport são duas coisas diferentes. Viewport é o “campo de visão” da sua tela. O viewport contém a sua câmera.

Stretch & Screen Viewport

touch point is working perfectly

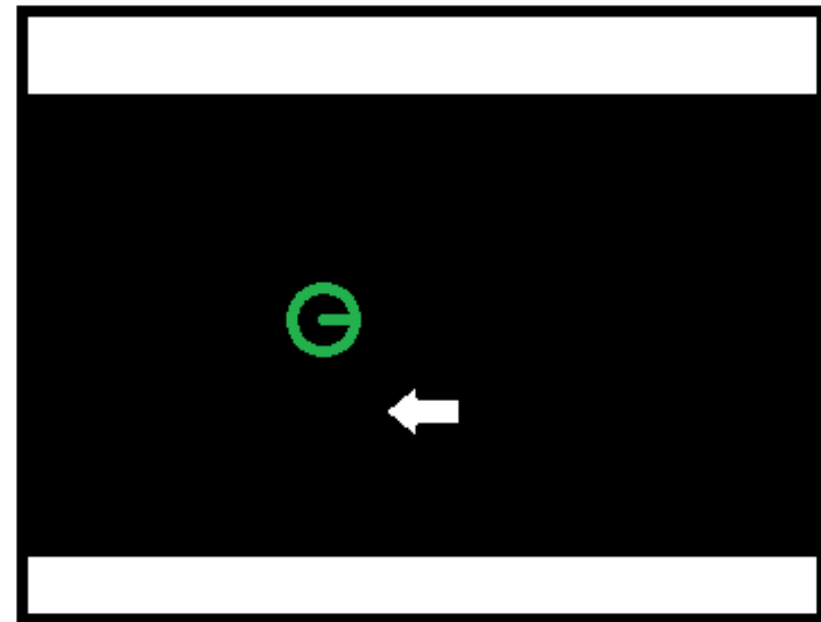


On Resize Screen

touch point is working perfectly

Fit Viewport

touch point is not behaving properly



On Resize Screen

touch point is working properly (so strange : ()



VIEWPORT

- Quando lidamos com diferentes tamanhos de tela/resoluções, é importante decidir como queremos que o conteúdo seja disponibilizado na tela.
- O viewport faz justamente isso, ajuda a definir uma resolução virtual, e é uma forma de tratar esse problema.



EXEMPLO DE USO DO VIEWPORT

- Um viewport pode ser aplicado da seguinte forma:

```
private Viewport viewport;  
private Camera camera;
```

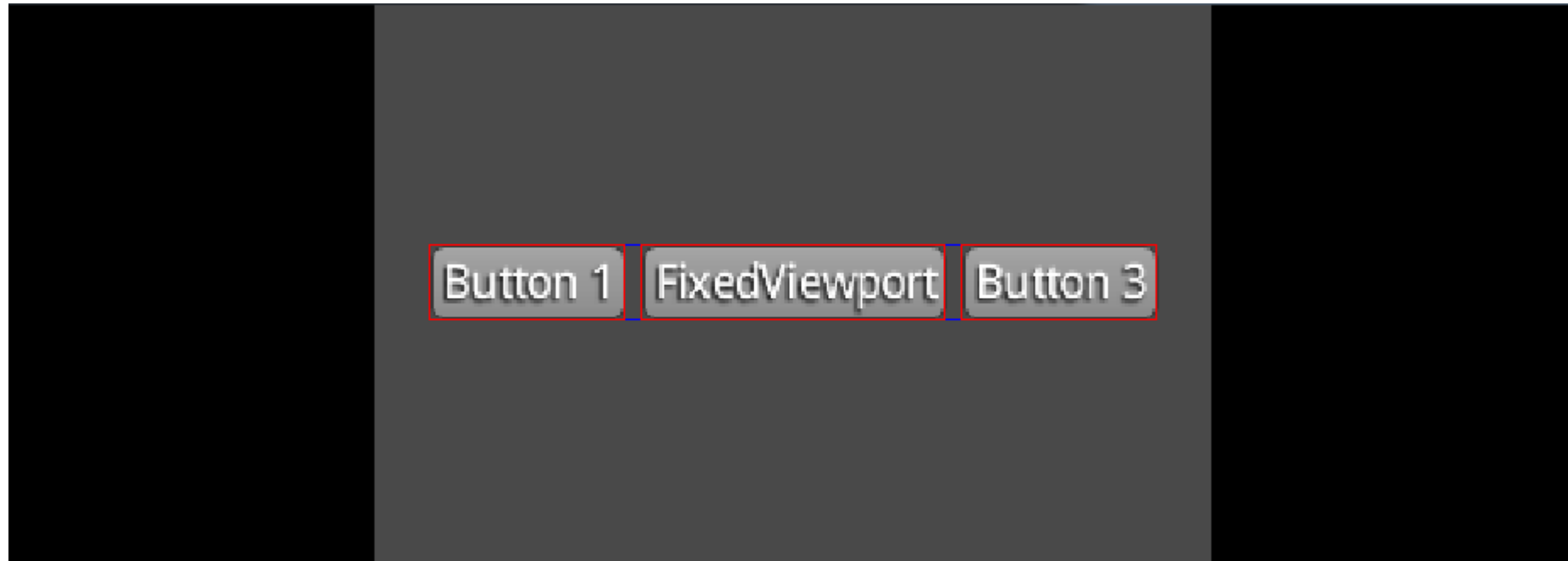
```
public void create() {  
    camera = new PerspectiveCamera();  
    viewport = new FitViewport(800, 480, camera);  
}
```

- Depois, quando a tela for redimensionada, você precisa chamar dentro do método `resize`:

```
public void resize(int width, int height) {  
    viewport.update(width, height);  
}
```

FIT VIEWPORT

- O **FitViewport** suporta uma resolução virtual de tela. A diferença de um **StretchViewport** (que veremos a seguir) é que ele não distorce as texturas originais.



STRETCH VIEWPORT

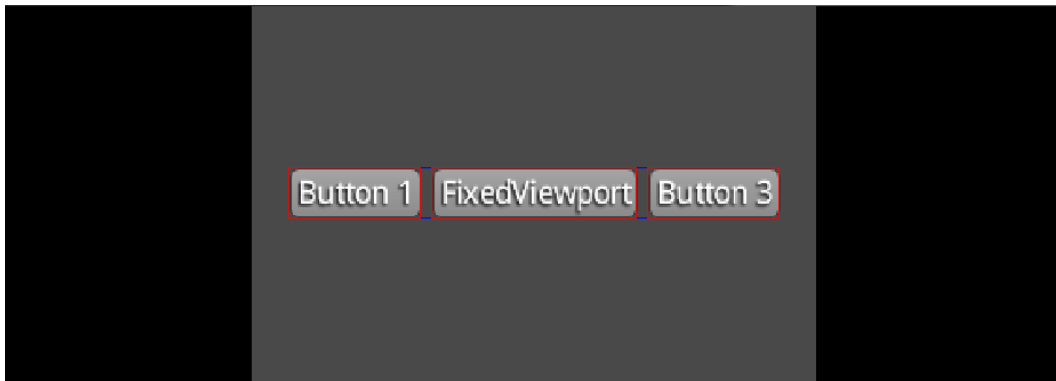
- O **StretchViewport** respeita a resolução da tela real. Nesse caso, **não existirão barras pretas** dos lados mas o **aspect ratio** (a divisão entre altura e largura) **será modificado e as texturas serão esticadas**.



FILL VIEWPORT

- O FillViewport vai preencher toda a tela e manter o aspect ratio (ou seja, não existirão as barras pretas). Contudo, nesse caso, parte do conteúdo pode acabar sendo cortado e ficando pra fora da tela.

- FitViewport

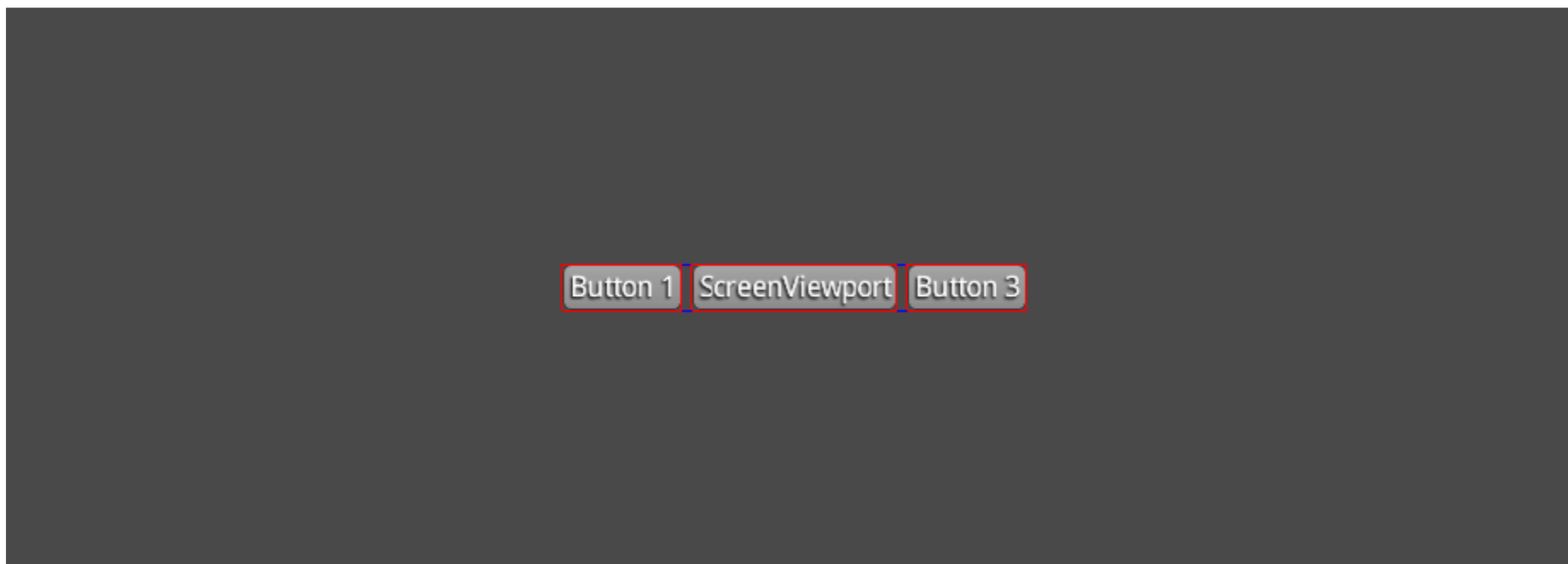


- FillViewport



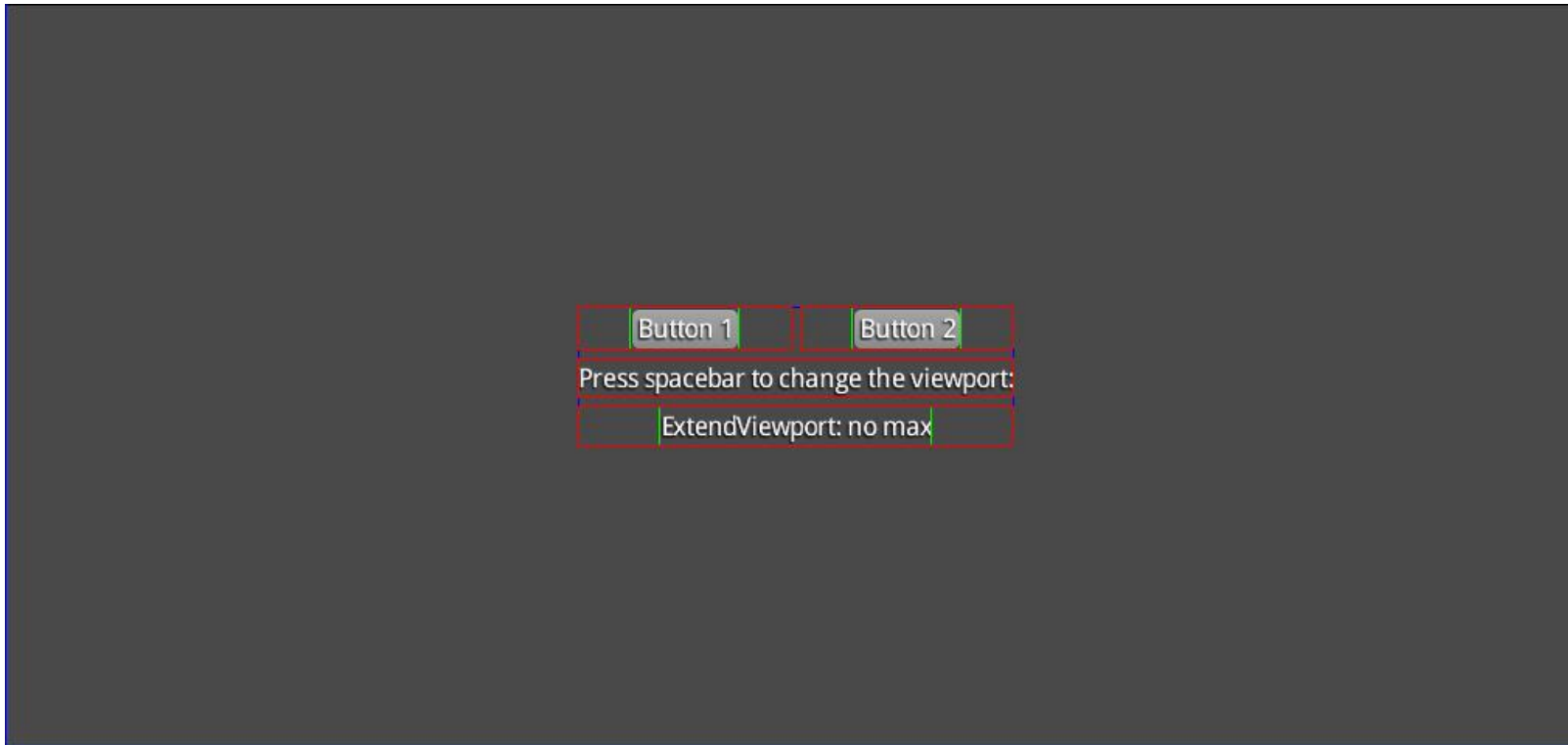
SCREEN VIEWPORT

- O ScreenViewport **não tem um tamanho de tela virtual**. Esse viewport vai sempre **respeitar a resolução original do cliente** e nenhuma distorção acontece.
 - Contudo, o gameplay pode mudar entre jogadores pois jogadores com uma resolução maior pode **ver mais do jogo do que um com resolução menor**.



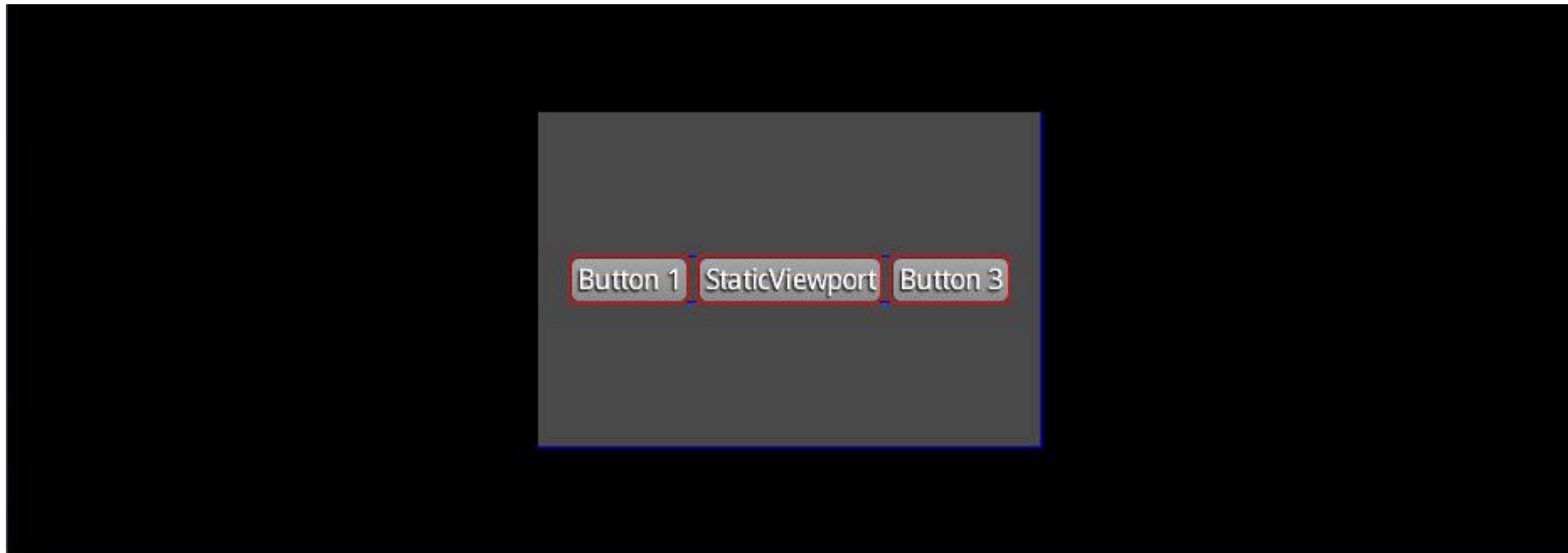
EXTEND VIEWPORT

- O ExtendViewport estende o viewport com relação a maior medida (altura e largura) pra evitar as barras pretas. Você passa as medidas e ele aumenta a dimensão (largura ou altura) que for menor.



CUSTOM VIEWPORT

- Também é possível **customizar um viewport** com tamanho e resolução que você preferir.





PERGUNTA

- Dado tudo isso que vimos até agora, a largura e altura da câmera podem ser **diferentes** da largura e altura do viewport?



PERGUNTA

- Dado tudo isso que vimos até agora, a largura e altura da tela de jogo podem ser **diferentes** da largura e altura do viewport e/ou da câmera?
 - Sim, podem!
 - Em algumas situações vão ser iguais e em algumas situações **serão diferentes**.
 - No exemplo do FitViewport, por exemplo, **as larguras podem ser diferentes**.
 - Camera, viewport e tela de jogo são **3 coisas diferentes** que não estão necessariamente relacionadas (não precisam ter exatamente a mesma resolução).



TELAS (GAME E SCREEN)

- Usualmente, a sua aplicação precisará de mais de uma tela. Em um jogo, por exemplo, podemos ter **tela de iniciar**, **tela do jogo**, **tela de game over**, etc.
 - Essas telas seriam mais ou menos representadas como uma **Activity** no Android.
- A libgdx implementa uma funcionalidade para troca fácil de telas.
 - Todas as telas terão uma **estrutura muito parecida** com a que já vimos, com os métodos: render, show, dispose, resize, etc.



CLASSE GAME

- Na sua classe principal, onde você lê:

```
public class MyGdxGame extends ApplicationAdapter {
```

- Mude para:

```
public class MyGdxGame extends Game {
```

- A partir desse momento a sua classe game te dá acesso a um método **setScreen**. Com esse método você pode **trocar a tela da classe principal (Game)** a qualquer momento.



SCREEN

- Screen é uma **interface**.
- Caso você queira criar uma nova tela você deve iniciar uma nova classe que implementa a interface Screen.

```
public class MinhaTela implements Screen
```

- Sua classe terá os **métodos**: (1) show, (2) render (**delta**), (3) resize(width, height), (4) pause, (5) resume, (6) hide e (7) dispose.
 - Veremos primeiramente o pause, resume e hide, que são métodos “novos”.

MÉTODOS DA SCREEN

- `pause()`:
 - Esse método é chamado quando a aplicação entra em **estado de pause**. Quando no android por exemplo o **usuário troca a aplicação**, a sua aplicação entrará nesse estado.
- `resume()`:
 - Esse método é chamado quando a tela **volta de um estado de pause**.
- `hide()`:
 - Esse método é chamado quando essa sua tela **deixa de ser a tela corrente de Game**. Ou seja, o método `setScreen` é chamado trocando a tela por uma outra tela. A tela anterior tem o seu método `hide` chamado.
- `resize(width, height)`:
 - **É chamado quando a tela é redimensionada**. Você recebe as novas dimensões (`width` e `height`) como parâmetro da função.



EXERCÍCIO

- Altere a sua aplicação principal para **Game**.
- Crie **duas telas** pelo menos:
 - Uma tela de jogo (pode ser a que você já tem, basta converter para implementar a interface Screen).
 - Uma tela de “boas-vindas” ou de início de jogo. Tela de menu.
- Faça a **conversão de uma tela pra outra por meio de um clique de botão**.
 - Ou, como ainda não vimos botão, vocês podem simplesmente colocar uma textura e ver se a coordenada do clique tá dentro dessa textura.



DELTA

- A partir do momento que trocamos a classe da aplicação para **Game** e começamos a utilizar as Screens, é possível perceber que agora o método **render** possui um novo parâmetro, o **delta**.
- Vamos fazer um experimento:
 - No método render coloque:

```
Gdx.graphics.setTitle(Gdx.graphics.getFramesPerSecond() + "");
```
 - **Execute** a sua aplicação.
 - Qual **FPS** você vê?



DELTA

- A partir do momento que trocamos a classe da aplicação para **Game** e começamos a utilizar as Screens, é possível perceber que agora o método **render** possui um novo parâmetro, o **delta**.
- Vamos fazer um experimento:
 - No método render coloque:

```
Gdx.graphics.setTitle(Gdx.graphics.getFramesPerSecond() + "");
```
 - **Execute** a sua aplicação.
 - Qual **FPS** você vê?
 - É bem provável que você tenha visto **no máximo 60 de fps**. Por quê?



FPS CONFIG

- Voltando na config do DesktopLauncher:

*/** Framerate quando a janela estiver em primeiro plano. A CPU vai dormir se for necessário. Use 0 para nunca deixar dormir.*

public int foregroundFPS = 60;

*/** Framerate quando a janela não estiver em primeiro plano. A CPU vai dormir se for necessário. Use 0 para nunca dormir e -1 para não renderizar. */*

public int backgroundFPS = 60;



DESTRAVAR FPS

- Para destravar o FPS você precisa fazer duas coisas.
 - Primeira, desativar o vsync (o vsync **limita seu FPS à taxa de atualização do seu monitor ou a 60 fps usualmente**):
`config.vSyncEnabled = false;`
 - Segundo, coloque o foregroundFPS em 0 (para não limitar).
`config.foregroundFPS = 0;`
- Agora, rodem a aplicação e façam um pequeno teste, **alternem a aplicação de plano de fundo / plano atual pra vocês verem a mudança no FPS** (no plano de fundo o fps vai estar limitado em 60 ainda).
 - Outra coisa, tentem **andar no seu jogo e veja se perceberam alguma mudança**.



DELTA

- Os movimentos estão muito mais rápidos, correto?
 - Isso acontece porque agora o FPS está destravado e o render está rodando a uma frequência / velocidade muito maior do que anteriormente.
 - Assim, no mesmo intervalo de tempo, o seu boneco **vai andar muito mais!**
- Bom, é exatamente pra isso que o delta serve! Para normalizar os movimentos e as animações em diferentes taxas de FPS multiplique todas as constantes de animação e movimentação do seu jogo por delta. Isso resolve o problema.



DELTA

- Se vocês já jogaram emuladores alguma vez na vida, existe uma forma de **acelerar** o jogo que é justamente aumentando ou destravando o FPS.
 - Esses jogos de console rodam em um framerate estável (**os consoles são padronizados**) e eles não precisam utilizar o delta por conta disso.
- No nosso caso, como a libgdx é multiplataforma e **não há como ter controle de quanto FPS o usuário final terá** (seja no celular, no pc, ou em qualquer outro lugar), é necessário utilizar a variável **delta**.
- A variável **delta** te dá o **tempo decorrido entre o frame anterior e o frame atual**.



EXERCÍCIO

- Corrija a movimentação do seu jogo em função do novo render com a **variável delta**.
- Limite novamente o FPS pra ver **se a movimentação dos elementos vai manter o mesmo padrão** (mesmo em FPS diferentes).

RENDERIZAÇÃO CONTÍNUA

- Por padrão, na libgdx, a **thread de renderização chama o método render() de forma contínua**, com uma frequência (quantidade de FPS) que depende da configuração inicial e do hardware.
- Se você fizer um jogo ou aplicação **meio estática como, por exemplo, um jogo de cartas**, você não precisa necessariamente do método **render rodando o tempo todo**.
 - Dessa forma, você pode **desativar a renderização contínua** e chamar o método render quando for necessário de fato desenhar alguma mudança.

RENDERIZAÇÃO CONTÍNUA

- Desativar o método render pode garantir um **menor consumo de energia em dispositivos com bateria**.
- Tudo que você precisa fazer é **adicionar no método create**:

```
Gdx.graphics.setContinuousRendering(false);  
Gdx.graphics.requestRendering();
```
- A primeira linha **para/encerra a renderização contínua e a segunda linha chama ou dispara o método render uma vez**. Você precisa utilizar a segunda linha nos momentos em que quiser que o método render seja chamado.

RENDERIZAÇÃO CONTÍNUA

- Se a renderização contínua **estiver desativada**, o método `render` será chamado apenas quando uma das seguintes coisas acontecer:
 - Um **evento de input é disparado** (clique do mouse, pressionamento de tecla, etc).
 - Quando `Gdx.graphics.requestRendering()` é chamada.
 - Quando `Gdx.app.postRunnable()` é chamado.